

DSA STUDY BLUEPRINT SERIES

71. Implement LRU Cache Linked List

Designing Least Recently Used Cache Cache Evictions

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Fast $O(1)$ access and item updates.
- **Data Structures:** Hash Map (stores keys & node pointers) + Double Linked List (tracks usage order).

Memory & Execution Blueprint

[MRU Node] ↔ [DLL Nodes] ↔ [LRU Node (To Evict)]

C++ Implementation Reference

Standard code layout and syntax

```
struct Node {  
    int key, val;  
    Node *prev, *next;  
    Node(int k, int v) : key(k), val(v), prev(nullptr), next(nullptr) {}  
};  
// Map stores unordered_map to query node links instantly.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
get / put Operations	$O(1)$ time, $O(\text{Capacity})$ space

Key Practices & Gotchas

- **Important:** Move queried nodes to the DLL head pointer location to update usage history.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace monotonic stack modifications for next greater element on `[4, 5, 2]` :

Active Element	Stack State (Indices)	Pop comparison	Result output
4	[0] (value 4)	N/A	Waiting for next greater
5	[1] (value 5)	5 > 4 (pop index 0)	Next Greater of 4 is 5

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Monotonic properties:** Stacks process elements strictly increasing or decreasing, optimizing nested lookups.
- **Min Stack details:** Track current minimum bounds by storing elements as encoded difference values.
- **Related LeetCode Problems:** #155 Min Stack, #84 Largest Rectangle in Histogram, #42 Trapping Rain Water.